

Значит надо вот что подумать. Первое расширение это конвертер веб страниц в pdf и pptx. Но вообще еще у нас будут гптс которые будут приводить трафик по задумке и веб сервис. В веб сервисе будет только функционал по ссылке. Текущую вкладку можно только через расширение конвертнуть. Также в веб сервисе должна быть промо страница, небольшой seo-блог. И думаю вот можно ли сделать на некстжс его чтобы предоставлять и нашим гптс ручки для конвертации? И seo там настроить

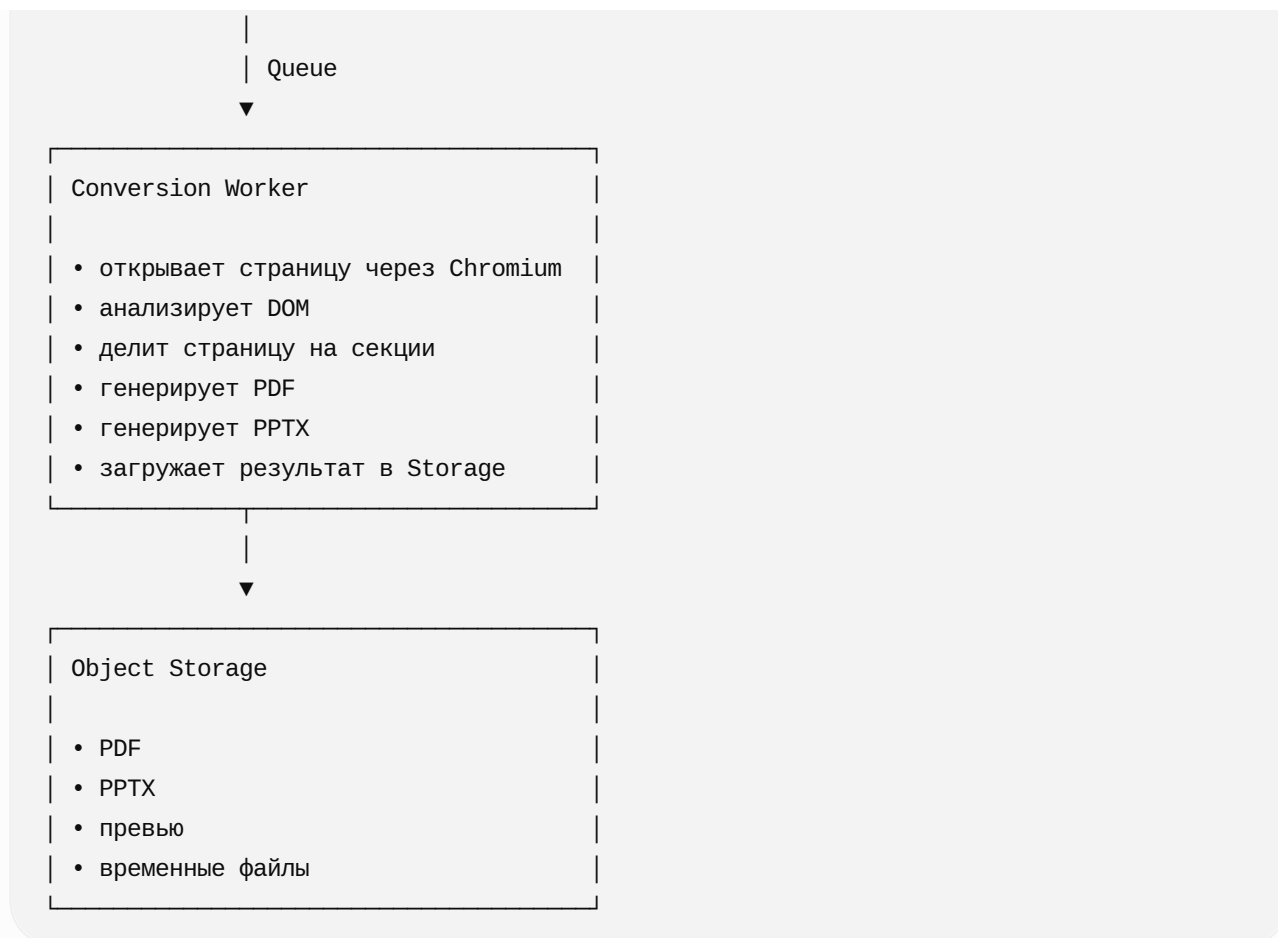
Обработка заняла 16s

Да, **Next.js** хорошо подходит как основа веб-сервиса, **SEO-сайта** и **API-шлюза для ваших GPTs**. Но самый тяжёлый процесс конвертации веб-страницы в PDF/PPTX лучше не выполнять непосредственно внутри Next.js-функции.

Оптимальная схема: **один продуктовый backend, несколько каналов привлечения и использования**.

Общая архитектура





Что именно делать на Next.js

Next.js App Router позволяет делать публичные HTTP-endpoints через Route Handlers, поэтому в одном приложении можно разместить и сайт, и API.

 Ne... +2

На Next.js я бы оставил:

- лендинг;
- страницы отдельных инструментов;
- SEO-блог;
- пользовательскую форму конвертации;
- личный кабинет в будущем;
- API для GPTs;
- API для расширения;
- создание задания на конвертацию;
- получение статуса задания;
- выдачу ссылки на готовый файл;
- rate limiting;
- аналитику источников трафика;
- управление бесплатными лимитами.

Но Next.js не должен внутри одного HTTP-запроса запускать Chromium, загружать сложный сайт, анализировать DOM и строить большой PPTX.

Технически это иногда работает, но архитектурно будет ненадёжно из-за:

- ограничений времени выполнения;
- потребления памяти Chromium;
- больших бинарников;
- нестабильной продолжительности загрузки сторонних сайтов;
- параллельных конвертаций;
- необходимости повторных попыток;
- возможных зависаний на тяжёлых страницах.

У Vercel Functions есть ограничения по продолжительности, размеру deployment bundle, памяти и другим ресурсам. На текущих тарифах длительность можно увеличивать, но это всё равно не превращает serverless-функцию в подходящий Chromium-worker.  Ve... +2

Правильная модель конвертации

API не ждёт, пока будет готов PDF. Оно создаёт задание:

http

POST /api/v1/conversions

Ответ:

JSON

```
{
  "jobId": "conv_8f2a1",
  "status": "queued",
  "statusUrl": "https://page2file.com/api/v1/conversions/conv_8f2a1"
}
```

Далее worker забирает задание из очереди.

После выполнения:

http

GET /api/v1/conversions/conv_8f2a1

JSON

```
{
  "jobId": "conv_8f2a1",
  "status": "completed",
}
```

```
"format": "pdf",  
"downloadUrl": "https://files.page2file.com/...",  
"expiresAt": "2026-07-30T08:00:00Z"  
}
```

Такой API смогут использовать:

1. веб-сервис;
2. Chrome-расширение;
3. GPTs;
4. в будущем публичный API;
5. внутренние административные инструменты.

Как GPTs будут использовать сервис

Да, ваши GPTs смогут обращаться к этому backend через **GPT Actions**.

Actions позволяют GPT подключаться к определённым вами внешнему API; API описывается через OpenAPI schema.  OpenAI Help... +1

Например пользователь пишет GPT:

Convert this page to a visual PDF:
<https://example.com/article>

GPT вызывает:

```
http  
  
POST https://api.page2file.com/v1/conversions
```

С параметрами:

```
JSON  
  
{  
  "url": "https://example.com/article",  
  "format": "pdf",  
  "mode": "visual",  
  "source": "gpt_page_to_pdf"  
}
```

После завершения GPT возвращает пользователю ссылку на файл.

Важное ограничение GPTs

GPT не сможет получить содержимое активной вкладки Chrome. У GPT нет доступа к текущей вкладке пользователя.

Поэтому разделение получается логичным:

Канал	По URL	Текущая вкладка
Веб-сервис	Да	Нет
GPT	Да	Нет
Chrome-расширение	Да	Да

И это можно использовать в воронке:

Для страниц, доступных по ссылке, используйте веб-конвертер.
Для авторизованных страниц, localhost, внутренних систем и текущей вкладки установите расширение.

Так расширение получает реальную ценность, которую невозможно полностью повторить веб-сервисом или GPT.

Как разделить API для GPT и расширения

Не стоит делать отдельную конвертационную систему для каждого GPT. Лучше общий API с указанием источника.

JSON

```
{
  "url": "https://example.com",
  "format": "pdf",
  "mode": "visual",
  "source": "gpt_web_to_pdf"
}
```

Источники:

```
website
chrome_extension
gpt_page_to_pdf
gpt_web_to_pdf
gpt_html_to_pdf
gpt_web_to_powerpoint
gpt_pdf_to_powerpoint
```

Это даст точную аналитику:

- сколько конвертаций приходит из каждого GPT;
- какие GPT приводят регистрации;
- какие форматы популярнее;

- сколько пользователей переходят на сайт;
- сколько устанавливают расширение;
- какой GPT имеет смысл продвигать.

Для GPT Actions можно сделать отдельный слой:

```
/api/gpt/v1/conversions
```

А для продукта:

```
/api/v1/conversions
```

Внутри они используют один и тот же сервис:

```
ConversionService.createJob()
```

Разделение endpoint'ов полезно для разных:

- лимитов;
- API-ключей;
- правил авторизации;
- схем ответов;
- аналитики;
- защиты от злоупотреблений.

Где запускать worker

Я бы рассматривал два варианта.

Вариант 1 — отдельный Node.js-сервис

```
Next.js: Vercel  
Worker: Railway / Render / Fly.io / Google Cloud Run  
Queue: Redis + BullMQ  
Storage: Cloudflare R2 / S3  
Database: PostgreSQL
```

Worker:

```
Node.js  
Playwright  
Chromium  
pdf-lib или Playwright PDF  
PptxGenJS  
Sharp  
BullMQ
```

Это наиболее понятная архитектура для MVP.

Вариант 2 — контейнерные задания

Next.js: Vercel
Jobs: Google Cloud Run Jobs / AWS ECS
Queue: Cloud Tasks / SQS
Storage: R2 / S3

Этот вариант лучше масштабируется, но для первого MVP сложнее.

Для начала я бы выбрал **отдельный постоянно запущенный Node.js worker + BullMQ**.

Структура Next.js-проекта

```
apps/  
  web/  
    app/  
      (marketing)/  
        page.tsx  
        pricing/  
        extension/  
        about/  
  
      tools/  
        web-to-pdf/  
        webpage-to-pdf/  
        html-to-pdf/  
        web-to-powerpoint/  
        webpage-to-pptx/  
  
      blog/  
        page.tsx  
        [slug]/  
  
      convert/  
        page.tsx  
        [jobId]/  
  
      api/  
        v1/  
          conversions/  
            route.ts  
            [jobId]/  
              route.ts  
  
      gpt/  
        v1/  
          conversions/
```

```
route.ts
openapi.json/
route.ts

sitemap.ts
robots.ts
manifest.ts

worker/
src/
  jobs/
  browser/
  analyzers/
  pdf/
  pptx/
  storage/
```

Можно начать с monorepo:

```
pnpm
Turborepo
TypeScript
```

И вынести общие пакеты:

```
packages/
  conversion-core/
  schemas/
  database/
  analytics/
  ui/
```

SEO на Next.js

Для такого продукта Next.js подходит хорошо. Он позволяет отдавать индексируемый HTML, создавать статические и серверные страницы, управлять metadata, canonical, sitemap и robots. В App Router есть отдельные Metadata API и файловые соглашения для `sitemap.xml` и

`robots.txt` .  Ne... +3

Главное — не сделать весь интерфейс одним клиентским React-приложением.

Индексируемые страницы должны рендериться на сервере

Например:


```
/web-to-pdf
/webpage-to-pdf
/html-to-pdf
/url-to-pdf
/save-webpage-as-pdf
```

```
/web-to-powerpoint
/webpage-to-pptx
/html-to-powerpoint
/url-to-pptx
```

Но это не должны быть почти одинаковые doorway pages. Для каждой страницы потребуется своя интент-ориентированная ценность:

- отдельный заголовок;
- отдельное описание сценария;
- собственные примеры;
- особенности формата;
- FAQ;
- ограничения;
- инструкция;
- связанные статьи;
- непосредственно работающий конвертер.

Например:

```
/web-to-pdf
```

Интент:

Convert any public webpage URL into a clean PDF.

Контент:

- форма URL;
- стандартный и visual режим;
- настройка размера страницы;
- удаление навигации;
- сохранение ссылок;
- FAQ.

```
/web-to-powerpoint
```

Интент:

Convert a webpage into an editable PowerPoint presentation.

Контент:

- объяснение разбиения сайта на слайды;
- редактируемые текстовые блоки;
- изображения отдельными объектами;
- выбор 16:9 или 4:3;
- примеры результата.

Google умеет обрабатывать JavaScript, но серверный или статический HTML уменьшает зависимость индексирования от клиентского рендеринга. Google отдельно отмечает, что JavaScript-сайты требуют учёта этапа рендеринга и возможных ограничений.  Google for D... +2

Рекомендуемая структура сайта

/	Главная
/web-to-pdf	Основной PDF-конвертер
/web-to-powerpoint	Основной PPTX-конвертер
/chrome-extension	Страница расширения
/blog	SEO-блог
/blog/how-to-save-a-webpage-as-pdf	
/blog/how-to-convert-a-webpage-to-powerpoint	
/blog/webpage-to-pdf-with-working-links	
/blog/convert-long-webpage-to-pdf	
/blog/convert-dashboard-to-powerpoint	
/privacy	
/terms	

На главной:

Convert any webpage to PDF or PowerPoint

Форма:

[Paste a webpage URL]

[PDF ▼] [Convert]

Ниже разделение:

Convert a public webpage by URL
No installation required

Need to convert the page currently open in your browser?
Install the Chrome extension

SEO-блог

Блог должен быть не новостным, а прикладным и связанным с конвертацией.

Кластеры:

Webpage to PDF

How to save a webpage as PDF
How to save a long webpage as one PDF
How to convert a webpage to PDF without ads
How to preserve clickable links in a PDF
How to convert an authenticated webpage to PDF
How to convert a webpage to PDF on Chrome

Webpage to PowerPoint

How to convert a webpage to PowerPoint
How to turn a website into presentation slides
How to convert HTML to editable PPTX
How to import website content into PowerPoint
How to convert a landing page into a presentation

Сравнительные и проблемные запросы

Print to PDF vs webpage converter
Why Chrome cuts off webpages when printing
How to fix broken page breaks in PDF
How to capture a dashboard as an editable presentation

Каждая статья ведёт:

- в онлайн-конвертер;
- на страницу расширения;
- в подходящий GPT.

Structured data

Для основных страниц можно использовать:

SoftwareApplication
WebApplication
FAQPage – только там, где FAQ действительно виден пользователю
Article – для блога
BreadcrumbList
Organization

Google поддерживает structured data для software applications, хотя разметка не гарантирует показ расширенного результата.  Google for D... +2

Критически важная безопасность

Поскольку backend будет открывать URL, введённый пользователем или GPT, нужно сразу защищаться от SSRF.

Запрещать обращения к:

```
localhost
127.0.0.0/8
10.0.0.0/8
172.16.0.0/12
192.168.0.0/16
169.254.0.0/16
::1
fc00::/7
метаданным облачных провайдеров
локальным именам хостов
file://
ftp://
data:
javascript:
```

Также нужны:

- повторная проверка IP после DNS resolution;
- проверка каждого redirect;
- ограничения размера страницы;
- таймаут загрузки;
- лимит количества ресурсов;
- блокировка скачивания файлов произвольного размера;
- изолированный Chromium;
- отключение доступа к локальной файловой системе;
- rate limits;
- captcha для анонимного веб-сервиса;
- отдельные ключи для каждого GPT;
- временные signed URLs для скачивания.

Без этого публичный URL-конвертер быстро превратится в инструмент сканирования внутренней сети.

Какую архитектуру я рекомендую окончательно

```
Next.js
└─ маркетинговый сайт
```

- └─ SEO-страницы
- └─ блог
- └─ веб-конвертер
- └─ API для расширения
- └─ API для GPT Actions
- └─ авторизация
- └─ лимиты
- └─ управление заданиями

PostgreSQL

- └─ users
- └─ conversion_jobs
- └─ api_clients
- └─ usage_events
- └─ files

Redis + BullMQ

- └─ очередь конвертаций

Node.js Worker

- └─ Playwright
- └─ DOM-анализатор
- └─ section detector
- └─ PDF generator
- └─ PPTX generator

Cloudflare R2

- └─ результаты
- └─ превью
- └─ временные ресурсы

Главный продуктовый вывод

Не следует воспринимать сайт, GPTs и расширение как три продукта с отдельной логикой.

Это должен быть **один conversion engine и три точки входа**:

GPTs → охватывают поисковые и разговорные сценарии

SEO-сайт → получает органический трафик из Google

Расширение → решает сложные сценарии текущей и закрытой вкладки

При этом:

- GPT возвращает результат и приводит человека на сайт;
- сайт даёт быструю конвертацию публичной ссылки;
- расширение конвертирует текущую вкладку, авторизованные страницы и локальные документы;
- всё работает через единый backend;

- аналитика показывает, какой канал и запрос реально приносит пользователей.

Итог: Next.js стоит использовать. Он подходит для веб-приложения, SEO, блога и ручек для GPT Actions. Но конвертационный движок следует вынести в отдельный worker, оставив Next.js роль сайта, API-шлюза и оркестратора заданий.

То есть бек на нод жс отдельно?

И еще момент, не должно быть сбора данных пользователей и регистрации и вообще если можно то итоговые файлы мы не храним на сервере, только локально

Да, но архитектуру стоит скорректировать под принцип **privacy-first**.

1. Нужен ли отдельный backend на Node.js

Для полноценной серверной конвертации — да. Но не обязательно делать его отдельным репозиторием или отдельным продуктом.

Можно использовать monorepo:

```
apps/  
  web/           Next.js: сайт, SEO, блог, API  
  worker/        Node.js: Chromium, PDF, PPTX  
  
packages/  
  converter-core/  
  schemas/  
  shared/
```

Физически:

- Next.js работает, например, на Vercel;
- Node.js worker работает в контейнере на Railway, Render, Fly.io или Cloud Run;
- код находится в одном репозитории;
- общего пользовательского аккаунта и базы пользователей нет.

Отдельный Node.js worker нужен не из-за языка: Next.js тоже работает на Node.js. Он нужен из-за **режима выполнения**. Chromium-конвертация тяжёлая, долгая и плохо подходит для обычных serverless-функций.

2. Без регистрации — нормально

Для первой версии можно полностью отказаться от:

- регистрации;
- аккаунтов;
- email;
- пользовательских профилей;
- истории конвертаций;
- постоянных идентификаторов;
- сохранения введённых URL;
- аналитики, связывающей действия с конкретным человеком.

Запрос может быть анонимным:

JSON

```
{
  "url": "https://example.com",
  "format": "pdf",
  "mode": "visual"
}
```

Для защиты от злоупотреблений всё равно потребуются технические меры:

- rate limiting по временному хешу IP;
- ограничение количества одновременных задач;
- максимальный размер страницы;
- таймаут;
- CAPTCHA только при подозрительной активности;
- защита от SSRF.

При этом сырой IP не обязательно записывать в базу. Можно вычислять короткоживущий хеш:

```
hash(ip + rotating_daily_secret)
```

Он автоматически перестаёт быть сопоставимым после смены секрета. Но даже это нужно честно описать в privacy policy как техническую обработку для защиты сервиса.

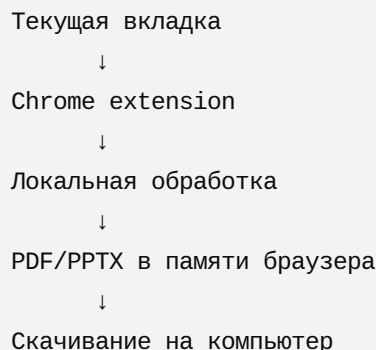
3. Можно ли вообще не хранить итоговые файлы

Здесь необходимо разделить три сценария.

Chrome-расширение: да, полностью локально

Для текущей вкладки расширение может:

1. прочитать DOM страницы;
2. собрать стили, текст, изображения и геометрию элементов;
3. сформировать PDF или PPTX в браузере;
4. сразу вызвать скачивание через Chrome Downloads API.



В таком режиме:

- HTML страницы не передаётся на сервер;
- файл не загружается на сервер;
- история не хранится;
- приватные и авторизованные страницы остаются на устройстве пользователя.

Это очень сильное позиционирование:

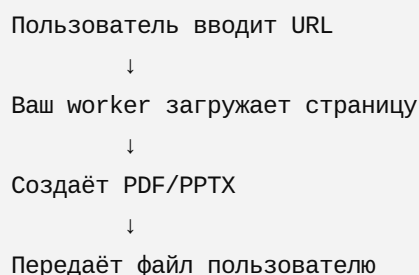
Your pages never leave your browser.

Однако есть техническая оговорка: сложная генерация PPTX с точным воспроизведением страницы может сильно нагружать браузер. Но для MVP это реализуемо, особенно если ограничивать размер страницы и заранее предупреждать о тяжёлых документах.

Веб-сервис по URL: частично локально или через временную серверную обработку

Когда пользователь вставляет URL на сайте, есть два подхода.

Подход А: сервер открывает URL



↓
Удаляет временные данные

Здесь страница и файл неизбежно обрабатываются сервером. Но их не обязательно **хранить постоянно**.

Можно использовать только:

- оперативную память;
- временный каталог контейнера;
- потоковую передачу ответа;
- удаление файла сразу после скачивания или завершения запроса.

Например:

```
/tmp/job-f7a91/output.pdf
```

После отправки:

```
unlink(output.pdf)
```

То есть корректное обещание:

Files are processed temporarily and are not permanently stored.

Но некорректно говорить:

Files never touch our servers.

Потому что при серверной конвертации они всё-таки создаются или проходят через сервер.

Подход В: браузер пользователя загружает URL сам

Теоретически Next.js-страница может попытаться загрузить внешний сайт и конвертировать его локально. На практике это будет плохо работать из-за:

- CORS;
- Content Security Policy;
- авторизации;
- защищённых изображений;
- cross-origin CSS;
- iframe-ограничений;
- антибот-защиты сайтов;
- невозможности корректно получить полный DOM произвольной страницы.

Поэтому универсальный веб-конвертер по URL почти неизбежно требует серверного Chromium.

4. Можно ли отдавать файл без Object Storage

Да. Для MVP отдельный S3 или Cloudflare R2 может не потребоваться.

Синхронная модель

```
http
```

```
POST /api/convert/pdf
```

Запрос ждёт завершения, затем получает файл:

```
http
```

```
Content-Type: application/pdf
```

```
Content-Disposition: attachment; filename="page.pdf"
```

Worker генерирует файл во временной папке и отправляет его потоком:

```
Chromium → temp file → response stream → delete
```

Минусы:

- долгое соединение;
- возможный timeout;
- сложно обрабатывать большие страницы;
- пользователь потеряет результат при обрыве соединения.

Асинхронная модель без постоянного хранения

1. POST /jobs
2. Worker создаёт файл во временном хранилище
3. GET /jobs/{id}/download
4. После первого скачивания файл удаляется
5. Если не скачан – удаляется через 10–30 минут

Это всё равно кратковременное хранение, но не постоянное.

Можно обозначить правило:

```
Максимальный TTL: 15 минут
```

```
Удаление после первого успешного скачивания
```

```
Нет пользовательской истории
```

Это наиболее практичный вариант для сайта.

5. Ограничение для GPTs

С GPTs ситуация отличается.

GPT вызывает ваш API удалённо. Он не может:

- запустить локальный код внутри браузера пользователя;
- сохранить бинарный файл прямо в локальную папку пользователя;
- использовать Chrome Downloads API;
- обратиться к вашему локальному расширению;
- получить доступ к текущей вкладке.

Следовательно, для GPT потребуется серверная конвертация.

Типичный поток:

```
GPT
↓
POST /api/gpt/conversions
↓
Worker создаёт файл
↓
Временная ссылка на скачивание
↓
GPT показывает ссылку пользователю
↓
Файл удаляется
```

Для GPT практически неизбежна временная доступность файла по URL. Причина: GPT должен получить некий результат, который сможет передать пользователю.

Рекомендуемая политика:

- файл доступен 10–15 минут;
- ссылка содержит криптографически случайный токен;
- файл удаляется после первого скачивания;
- ссылка не индексируется;
- имя файла не содержит исходный URL;
- никаких постоянных записей в базе;
- GPT получает только статус и временный URL.

Пример:

JSON

```
{  
  "status": "completed",  
  "downloadUrl": "https://api.example.com/dl/7bQx...Kp9",  
  "expiresIn": 900  
}
```

6. Рекомендуемая итоговая архитектура

Расширение

Текущая вкладка:

100% локальная обработка

100% локальное скачивание

Сервер не используется

При конвертации URL через расширение можно дать два режима:

Private local mode

Работает с текущей открытой вкладкой

URL mode

Публичный URL обрабатывается сервером временно

Но лучше для ясности оставить в расширении главным именно текущую вкладку.

Веб-сервис

Next.js:

- SEO
- лендинг
- блог
- форма
- API gateway
- статус задания

Node.js worker:

- загрузка URL
- Chromium
- PDF/PPTX
- временный файл
- потоковая выдача
- немедленное удаление

GPT Action

↓

Next.js API

↓

Node.js worker

↓

Временный файл до 15 минут

↓

Одноразовая ссылка

7. Нужна ли база данных

Для MVP можно обойтись без полноценной PostgreSQL.

Для короткоживущих заданий использовать Redis:

```
job:{id}
{
  status,
  format,
  createdAt,
  expiresAt,
  downloadToken
}
```

TTL:

15 минут

После истечения Redis сам удалит запись.

Файлы:

- временная директория worker;
- либо temporary bucket с lifecycle rule;
- без постоянного Object Storage.

Но если worker запускается как ephemeral container, пользователь может попасть на другой экземпляр при скачивании. Поэтому есть два варианта:

1. worker сразу стримит результат в одном соединении;
2. временный bucket с автоматическим удалением через короткий срок.

Для устойчивой асинхронной системы второй вариант надёжнее. При этом это всё равно **временная техническая обработка**, а не сохранение пользовательских файлов.

8. Как сформулировать privacy-позиционирование

Для расширения:

Private by design. When converting the current tab, processing happens locally in your browser. Page content and generated files are not uploaded to our servers.

Для сайта:

Public webpage URLs are processed temporarily to generate your file. Generated files are automatically deleted after download or within 15 minutes. We do not provide accounts or retain conversion history.

Для GPT:

Files generated through the GPT are temporarily available through an expiring download link and are automatically deleted.

Не стоит объединять это в заявление «всё работает локально», потому что для веб-сервиса и GPT это технически неверно.

Итоговое решение

Backend на Node.js нужен отдельно от Next.js runtime, но может находиться в том же моногеро.

Самая правильная модель:

Компонент	Где конвертируется	Хранение
Текущая вкладка в расширении	Локально в браузере	Только у пользователя
Публичная ссылка на сайте	В Node.js worker	Временно, до скачивания
Публичная ссылка через GPT	В Node.js worker	Временно, по одноразовой ссылке
Аккаунты	Не нужны	Нет
История конвертаций	Не нужна	Нет
Постоянное хранение файлов	Не нужно	Нет

То есть продукт может быть действительно privacy-first, но обещание должно быть точным: **локально для расширения, временно и без сохранения — для сайта и GPTs.**

Хорошо предпросмотр давай добавим что для пдф что для пптх, но историю не храним. И так

Бекенд должен умным образом уметь конвертировать точь в точь разного уровня страницы так чтобы сибирать дизайн и контент текстом, кнопками и ссылки кликабельные были. Также он должен уметь просто скриншотить сайт по секциям и закидывать в пдф картинками страницу. То же самое касаемо и презентаций. В двух режимах.

Show more

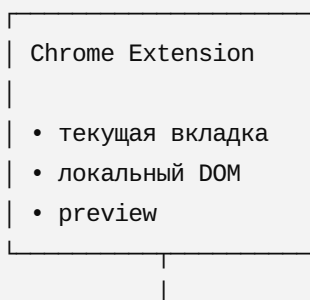
Обработка заняла 16s

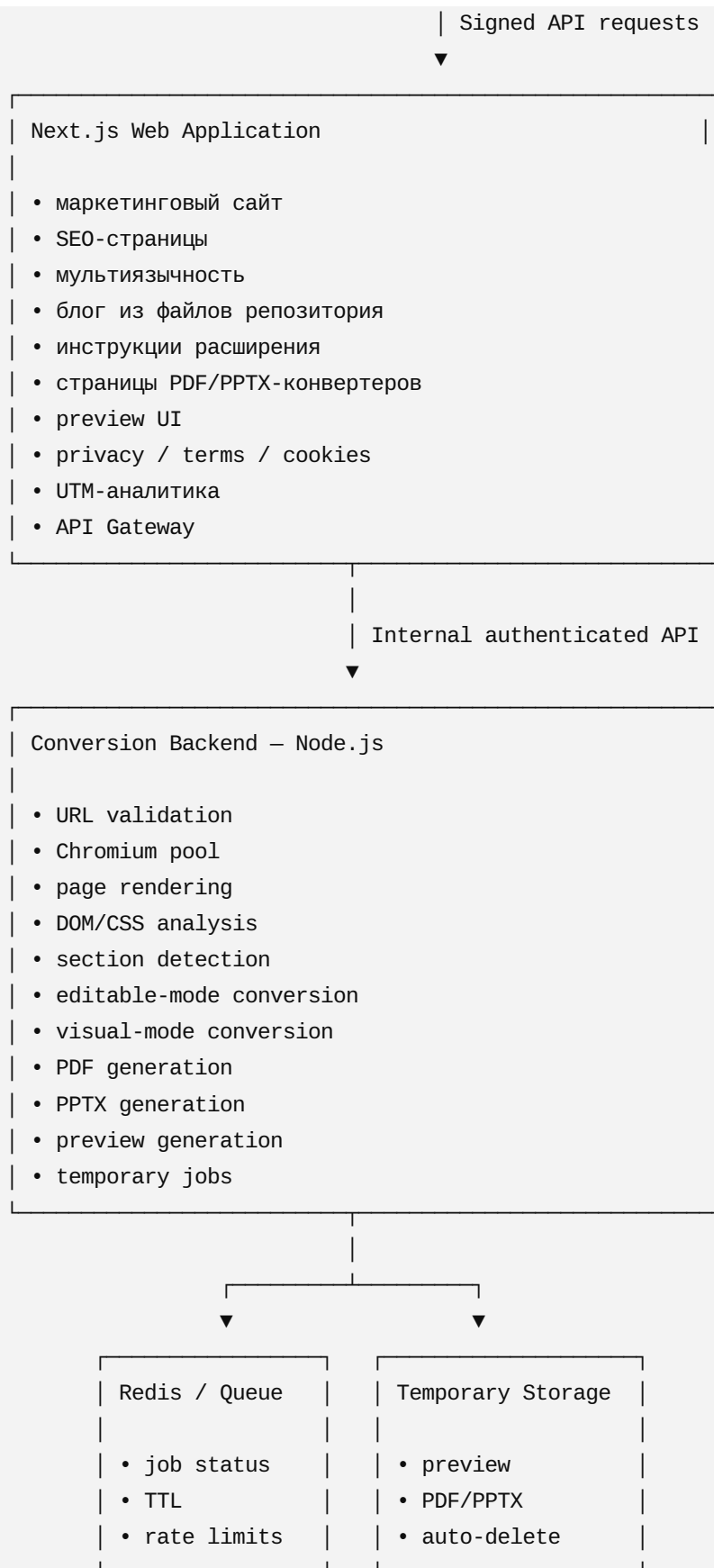
Да. Ниже — целевая архитектура с учётом всех требований: два режима конвертации, предпросмотр, отсутствие регистрации и истории, мультязычное SEO, GPT Actions, расширение и закрытый conversion API.

Главное архитектурное решение:

Next.js отвечает за сайт, SEO, интерфейс и API-шлюз. Отдельный **Node.js Conversion Backend** отвечает за Chromium, анализ страниц, PDF/PPTX, предпросмотр и временные файлы.

1. Общая архитектура продукта





Один monorepo

```
page2file/
├─ apps/
│   └─ web/           # Next.js
│   └─ api/           # Node.js API Gateway / orchestration
```



```
|   ├── worker/                # Chromium conversion worker
|   └── extension/            # Chrome extension
|
| ├── packages/
|   ├── conversion-core/
|   ├── page-analyzer/
|   ├── pdf-generator/
|   ├── pptx-generator/
|   ├── shared-schemas/
|   ├── security/
|   ├── analytics/
|   └── ui/
|
| ├── content/
|   ├── blog/
|   ├── changelog/
|   └── translations/
|
| └── infrastructure/
|     ├── docker/
|     ├── terraform/
|     └── monitoring/
```

Backend может быть отдельным приложением, но не обязательно отдельным репозиторием.

2. Два режима конвертации

На страницах конвертации пользователь выбирает не формат, а только режим.

Формат уже определяется страницей:

```
/web-to-pdf
/web-to-powerpoint
```

Режим 1: Visual

Это максимально точное визуальное воспроизведение.

```
Visual mode
```

Механика:

1. Chromium открывает страницу.
2. Дождидается загрузки шрифтов, изображений и динамического контента.

3. Определяет логические секции.
4. Делает снимок каждой секции.
5. Вставляет снимки в PDF-страницы или PPTX-слайды.
6. Поверх снимков при необходимости добавляет кликабельные области ссылок.

PDF

Каждая секция:

- становится отдельной PDF-страницей;
- либо несколько коротких секций объединяются;
- изображения сохраняют визуальную точность;
- ссылки можно сделать кликабельными через невидимые link annotations.

PPTX

Каждая секция:

- становится отдельным слайдом;
- фон слайда — изображение секции;
- ссылки могут накладываться прозрачными кликабельными областями.

Плюсы:

- максимальная визуальная точность;
- хорошо работает со сложными лендингами;
- сохраняет градиенты, тени, анимационные состояния и нестандартный CSS;
- меньше проблем со шрифтами.

Минусы:

- текст нельзя полноценно редактировать;
- кнопки визуальные, а не нативные объекты;
- файл может быть тяжелее.

Режим 2: Editable / Clickable

Название `Clickable` не полностью описывает режим. Лучше назвать его:

Editable

А в описании:

Editable text, buttons and clickable links.

Механика:

1. Chromium открывает и стабилизирует страницу.
2. Backend получает DOM, computed styles и геометрию элементов.
3. Страница делится на секции.
4. Каждый значимый элемент классифицируется:
 - текст;
 - изображение;
 - кнопка;
 - ссылка;
 - фон;
 - контейнер;
 - иконка;
 - декоративный объект.
5. Элементы собираются заново в PDF или PPTX.

Что сохраняется нативно

- текст отдельными текстовыми блоками;
- ссылки кликабельными;
- кнопки отдельными shape-объектами;
- изображения отдельными изображениями;
- фон отдельным элементом;
- простые рамки, скругления, заливки и тени;
- расположение и размеры элементов;
- базовая типографика.

Что может растриваться

Сложные элементы:

- CSS masks;
- backdrop-filter;
- сложные SVG-фильтры;
- WebGL;
- canvas;
- видео;
- сложные псевдоэлементы;
- нестандартные blend modes;
- тяжёлые декоративные композиции.

Для них используется гибридный подход:

Важное ограничение точности

Требование «точь в точь и всё редактируемое» нельзя гарантировать одновременно для любой существующей веб-страницы.

Причина: модели HTML/CSS, PDF и PowerPoint существенно различаются.

Поэтому backend должен использовать **adaptive fidelity strategy**:

Если элемент можно воспроизвести нативно → нативный объект

Если нельзя воспроизвести точно → растровый fallback

Если весь блок слишком сложный → снимок блока

Именно так можно получить максимально точный результат без разрушения дизайна.

3. Умный движок анализа страницы

Этап 1. Загрузка

Worker использует Playwright и Chromium.

```
URL
↓
URL Security Validator
↓
Isolated Chromium Context
↓
Page Stabilizer
```

Page Stabilizer:

- ждёт `document.fonts.ready`;
- ждёт загрузки изображений;
- выполняет ограниченный controlled scroll для lazy-loading;
- ждёт затухания сетевой активности;
- отключает бесконечные анимации;
- останавливает autoplay;
- закрывает или пытается нейтрализовать cookie banners;
- устанавливает заданный viewport;
- фиксирует финальное состояние страницы.

Этап 2. Анализ DOM

Для каждого элемента собирается:

TypeScript

```
interface ExtractedElement {  
  tagName: string;  
  role?: string;  
  text?: string;  
  href?: string;  
  boundingBox: Rect;  
  computedStyle: NormalizedStyle;  
  zIndex: number;  
  opacity: number;  
  transform?: Matrix;  
  background?: BackgroundDefinition;  
  children: ExtractedElement[];  
}
```

Этап 3. Семантическая классификация

Используются:

- HTML-теги;
- ARIA roles;
- computed styles;
- размеры;
- пространственное расположение;
- визуальная иерархия;
- повторяемость элементов;
- плотность текста;
- контраст;
- DOM-дерево.

Категории:

```
header  
hero  
feature-section  
content-section  
pricing  
testimonial  
gallery  
table  
footer  
navigation  
modal
```

Для MVP классификация должна быть преимущественно алгоритмической, а не LLM-зависимой. ИИ можно использовать как необязательный fallback для определения секций, но базовый конвертер не должен зависеть от стоимости и задержки внешней модели.

Этап 4. Деление на секции

Секция определяется по совокупности:

- `<section>`, `<article>`, `<header>`, `<footer>`;
- крупные вертикальные интервалы;
- смена фона;
- горизонтальные границы;
- логические контейнеры;
- заголовки;
- резкие изменения плотности контента;
- viewport-sized blocks;
- ограничения размера PDF-страницы или PPTX-слайда.

Движок должен уметь:

- не разрывать заголовок и следующий абзац;
- не разрезать кнопку;
- не разрывать изображение;
- переносить слишком длинную секцию;
- объединять слишком короткие секции;
- учитывать sticky и fixed elements;
- исключать плавающие чаты и cookie banners.

4. Генерация PDF

Visual PDF

Section screenshots

↓

Image optimization

↓

PDF page layout

↓

Clickable link overlays

↓

Final PDF

Editable PDF

PDF не является редактором объектов в том же смысле, что PowerPoint, но можно сохранить:

- настоящий выделяемый текст;
- кликабельные ссылки;
- векторные прямоугольники;
- кнопки;
- изображения;
- фоновые элементы;
- аннотации.

Подход:

PDFKit / pdf-lib / custom renderer

Для сложных страниц:

Native PDF objects + rasterized fallback layers

5. Генерация PPTX

Visual PPTX

- секция = слайд;
- изображение секции = фон;
- прозрачные link overlays;
- пропорции 16:9 по умолчанию;
- возможность позже добавить 4:3.

Editable PPTX

Для генерации:

PptxGenJS + custom layout engine

Преобразование:

HTML text	→ PPTX text box
button	→ shape + text + hyperlink
image	→ image object
background	→ shape or image
border	→ line/shape

link	→ hyperlink
SVG	→ SVG image or converted vector
complex block	→ raster image

Шрифты:

- если шрифт доступен и совместим — сохраняется имя;
- если отсутствует — подбирается fallback;
- веб-шрифты нельзя просто распространять внутри PPTX без проверки лицензии;
- предпросмотр должен предупреждать о font substitution.

6. Предпросмотр

Предпросмотр нужен для обоих форматов и режимов.

Пользовательский поток

1. Вставить URL
2. Выбрать Visual или Editable
3. Нажать Generate preview
4. Посмотреть секции/слайды
5. При необходимости:
 - удалить секцию;
 - изменить порядок;
 - объединить;
 - разделить;
6. Нажать Download PDF / Download PPTX

При этом формат скачивания уже фиксирован страницей.

Preview не должен быть самым итоговым файлом

Backend сначала создаёт нормализованную модель документа:

TypeScript

```
interface ConversionDocument {  
  width: number;  
  sections: ConversionSection[];  
  metadata: {  
    title?: string;  
    sourceUrl: string;  
  };  
}
```

Из неё генерируются:

- JPEG/WebP-превью;
- PDF;
- PPTX.

Это позволяет не конвертировать страницу полностью повторно после перестановки секций.

Временная жизнь preview

Job TTL: 15–30 минут

Preview TTL: 15–30 минут

Final file TTL: до первого скачивания или максимум 15 минут

История не хранится.

После:

- скачивания;
- истечения TTL;
- закрытия job;
- ошибки;

файлы и данные задания удаляются.

7. Структура сайта на Next.js

```
/[locale]
/[locale]/convert-webpage-to-pdf
/[locale]/convert-webpage-to-powerpoint

/[locale]/chrome-extension
/[locale]/chrome-extension/how-to-use

/[locale]/page2pdf-gpt
/[locale]/web2pdf-gpt
/[locale]/html2pdf-gpt
/[locale]/web2powerpoint-gpt

/[locale]/blog
/[locale]/blog/[slug]

/[locale]/updates
/[locale]/changelog

/[locale]/privacy
```

Главная страница

Содержит:

- краткое позиционирование;
- два главных действия:
 - Convert webpage to PDF;
 - Convert webpage to PowerPoint;
- объяснение двух режимов;
- преимущества расширения;
- privacy-first блок;
- ссылки на GPTs;
- примеры;
- FAQ;
- ссылки на блог.

Welcome page расширения

/[[locale]]/chrome-extension

Открывается после установки.

Содержит:

- приветствие;
- основные возможности;
- объяснение локальной и серверной обработки;
- кнопку «Open extension»;
- инструкцию;
- FAQ;
- ссылки на privacy policy;
- версию расширения;
- ссылку на обновления.

Инструкция расширения

/[[locale]]/chrome-extension/how-to-use

Переключатель:

[Step-by-step] [Video]

Step-by-step

- шаги;
- скриншоты;
- короткие пояснения;
- troubleshooting.

Video

- встроенное видео;
- текстовая расшифровка;
- главы;
- poster image.

Текстовая инструкция должна оставаться в HTML даже при наличии видео, чтобы страница индексировалась.

8. Страницы GPTs

Посадочные страницы должны соответствовать разным интендам.

Page2PDF GPT

```
/[locale]/page2pdf-gpt
```

Назначение:

- одна конкретная веб-страница;
- PDF;
- быстрый сценарий;
- ссылка на GPT;
- форма на сайте как альтернатива.

Web2PDF GPT

```
/[locale]/web2pdf-gpt
```

Назначение:

- многостраничный сайт;
- список URL;
- crawl по внутренним ссылкам;
- один PDF или ZIP из PDF;
- лимиты глубины и количества страниц.

Это технически отдельная задача от одной страницы. Для неё нужен crawler layer:

```
Start URL
↓
Same-origin crawler
↓
URL normalization
↓
Duplicate removal
↓
Per-page jobs
↓
Merge / ZIP
```

Обязательно:

- максимальное количество страниц;
- максимальная глубина;
- только тот же домен;
- соблюдение разумной нагрузки;
- запрет бесконечных URL-паттернов;
- защита от calendar traps;
- ограничение query parameters.

HTML2PDF GPT

```
/[locale]/html2pdf-gpt
```

Необходимо заранее определить, что принимается:

- сырой HTML;
- HTML + inline CSS;
- публичная ссылка на HTML;
- ZIP пока лучше не поддерживать в первом GPT.

HTML выполняется в изолированном sandbox без доступа к внутренней сети.

Web2PowerPoint GPT

```
/[locale]/web2powerpoint-gpt
```

Назначение:

- одна страница или ограниченный набор страниц;
- Visual PPTX;

- Editable PPTX;
- отдельные слайды по секциям.

9. Блог внутри репозитория

Отдельная CMS не нужна.

```
content/blog/  
├─ how-to-save-a-webpage-as-pdf.mdx  
├─ how-to-convert-webpage-to-powerpoint.mdx  
├─ webpage-to-pdf-with-clickable-links.mdx  
└─ visual-vs-editable-conversion.mdx
```

Формат:

```
MDX + frontmatter
```

Пример:

YAML

```
title: "How to Save a Webpage as a PDF"  
description: "..."  
publishedAt: "2026-08-01"  
updatedAt: "2026-08-01"  
locale: "en"  
translationKey: "save-webpage-as-pdf"
```

Не больше 10 статей — это нормально. Главное, чтобы каждая статья отвечала отдельному поисковому интенту и не дублировала посадочные страницы.

10. Страница обновлений и changelog

Лучше разделить:

```
/updates  
/changelog
```

Updates

Человекочитаемые продуктовые обновления:

- новые функции;

- скриншоты;
- объяснение пользы;
- ссылки на расширение.

Changelog

Технический список:

v1.2.0
Added:
Fixed:
Improved:

Файлы:

content/changelog/*.mdx

11. Мультиязычность и SEO

Next.js App Router поддерживает locale-based routing и серверный рендеринг локализованного контента. Sitemap можно генерировать программно через `sitemap.ts`.  Ne... +1

URL-структура

```
/en/convert-webpage-to-pdf
/de/convert-webpage-to-pdf
/fr/convert-webpage-to-pdf
/es/convert-webpage-to-pdf
/nl/convert-webpage-to-pdf
```

Даже английский должен иметь `/en`, чтобы структура была однородной.

Не «язык для каждой западной страны»

Язык и страна — разные сущности. Не нужно создавать отдельные версии для США, Канады, Австралии и Великобритании, если контент почти одинаковый.

Первая волна:

```
en
de
fr
```

es
pt
it
nl
pl

Вторая волна:

sv
no
da
fi
cs
ro
hu

Не следует сразу запускать 15–20 языков машинным переводом без проверки. Плохо переведённые страницы могут снизить доверие и создать сотни слабых SEO-страниц.

Для каждой локализованной страницы

Обязательно:

- настоящий переведённый контент;
- локализованные `title` и `description`;
- `<html lang="">` ;
- `canonical`;
- `hreflang` ;
- `x-default` ;
- локализованный Open Graph;
- отдельная запись в sitemap;
- одинаковый translation key;
- отсутствие автоматического редиректа поискового робота по IP.

Пример:

HTML

```
<link rel="alternate" hreflang="en" href="https://site.com/en/..." />
<link rel="alternate" hreflang="de" href="https://site.com/de/..." />
<link rel="alternate" hreflang="x-default" href="https://site.com/en/..." />
```

Индексация

Индексируются:

- главная;

- конвертеры;
- GPT landing pages;
- инструкция;
- блог;
- updates;
- changelog;
- privacy и terms при необходимости.

Не индексируются:

```
/preview/*  
/jobs/*  
/download/*  
/api/*
```

Через:

```
X-Robots-Tag: noindex, nofollow
```

и robots rules.

12. Google Analytics, UTM и cookie consent

Поскольку подключается Google Analytics, пользователь должен получать понятное согласие там, где этого требует применимое законодательство.

По умолчанию

До согласия:

```
analytics_storage = denied  
ad_storage = denied
```

После согласия:

```
analytics_storage = granted
```

Не следует загружать полноценные аналитические cookies до выбора пользователя в регулируемых регионах.

Cookie banner

Варианты:

Accept analytics
Reject non-essential
Manage preferences

Отказ должен быть таким же доступным, как согласие.

UTM-параметры

Поддерживать:

```
utm_source  
utm_medium  
utm_campaign  
utm_content  
utm_term
```

Например:

```
utm_source=page2pdf_gpt  
utm_medium=gpt  
utm_campaign=gpt_launch
```

Для расширения:

```
utm_source=chrome_extension  
utm_medium=extension  
utm_campaign=welcome
```

UTM можно:

- передавать в Google Analytics после согласия;
- хранить кратковременно в sessionStorage;
- не связывать с пользовательским профилем;
- не создавать собственную постоянную историю.

Что всё равно попадает в инфраструктурные логи

Полностью заявлять «никакие данные не собираются» нельзя, если используются:

- хостинг;
- CDN;
- firewall;
- Google Analytics;
- error monitoring;
- server access logs.

Нужно минимизировать:

- URL в логах;
- query parameters;
- IP retention;
- request bodies;
- исходный HTML;
- имена файлов.

13. Privacy, Terms и Cookie Policy

Да, нужны минимум:

Privacy Policy
Terms of Service
Cookie Policy

Также полезно:

Security
Copyright / Acceptable Use

Privacy Policy

Должна объяснять:

- что собирает Google Analytics;
- что происходит с UTM;
- что URL обрабатывается сервером;
- что файлы временные;
- TTL;
- что история не хранится;
- чем отличается локальная обработка расширения;
- какие технические логи могут существовать;
- какие сторонние сервисы используются;
- как удалить cookie;
- контакт для privacy-запросов.

Terms

Должны запрещать:

- конвертацию материалов без прав;
- злоупотребление crawler;

- сканирование внутренних сетей;
- обход авторизации;
- вредоносный HTML;
- чрезмерную автоматизацию;
- нарушение чужих прав.

14. Безопасность входных URL

Главная угроза сервиса — не SQL injection, а **SSRF**, потому что сервер открывает введённый пользователем URL. OWASP прямо выделяет риск доступа через такой endpoint к внутренним сервисам и cloud metadata.

 OWASP Chea... +1

URL Validator

Разрешены только:

https://
http:// – опционально, лучше ограничить

Запрещены:

file:
data:
javascript:
ftp:
blob:
chrome:
about:

Запрещённые адреса:

localhost
127.0.0.0/8
0.0.0.0/8
10.0.0.0/8
172.16.0.0/12
192.168.0.0/16
169.254.0.0/16
::1
fc00::/7
fe80::/10
cloud metadata endpoints

Проверять необходимо:

1. исходный hostname;
2. результат DNS resolution;
3. все IP-адреса;
4. каждый redirect;
5. повторный DNS lookup перед соединением;
6. IPv4-mapped IPv6;
7. нестандартные записи IP;
8. DNS rebinding.

Изоляция Chromium

Worker запускается:

- в отдельном контейнере;
- без доступа к приватной сети;
- без cloud metadata;
- с read-only filesystem, кроме `/tmp`;
- без привилегий;
- с ограниченными CPU и RAM;
- с ограниченным временем;
- с ограниченным объёмом скачиваемых ресурсов.

Инъекции

Так как принимается URL и, возможно, HTML:

- не выполнять shell-команды из пользовательских строк;
- не строить команду Chromium через строковую конкатенацию;
- использовать schema validation;
- экранировать HTML в preview UI;
- использовать CSP;
- не применять `dangerouslySetInnerHTML` к непроверенному содержимому;
- очищать SVG;
- запрещать произвольные script callbacks;
- не десериализовать неизвестные объекты;
- использовать prepared statements, если появится SQL;
- применять CSRF-защиту к state-changing web requests. OWASP рекомендует CSRF-токены для таких запросов, когда защита не обеспечивается платформой автоматически. [🔗 OWASP Cheat...](#)

15. Высокая нагрузка и скорость

Queue-based architecture

```
API Gateway
  ↓
Redis Queue
  ↓
Worker Pool
  ↓
Chromium Pool
```

Инструменты:

```
BullMQ
Redis
Playwright
Node.js
```

Отдельные очереди

```
preview-visual
preview-editable
render-pdf
render-pptx
crawl-site
```

Приоритеты:

```
preview > final render > multi-page crawl
```

Chromium pool

Не запускать новый браузер для каждого запроса.

```
Worker
├─ Chromium process
│   └─ Browser context 1
│       └─ Browser context 2
│           └─ Browser context 3
```

После определённого количества задач процесс перезапускается для очистки памяти.

Масштабирование

Worker масштабируется горизонтально:

Queue depth grows



Add worker instances



Queue stabilizes



Remove excess workers

Метрики:

- queue wait time;
- conversion duration;
- Chromium crash rate;
- memory per job;
- page load time;
- output size;
- preview time;
- success rate по режимам;
- timeout rate;
- fallback-to-raster rate.

Кэширование

Поскольку история не хранится, постоянный кэш пользовательских результатов не нужен.

Допустим только кратковременный технический кэш:

same normalized URL + same options

TTL 1-5 минут

Но для privacy-first версии его можно вообще отключить на старте.

16. API и вопрос «только наши клиенты»

Здесь важное ограничение:

Нельзя надёжно спрятать постоянный API-ключ внутри публичного сайта или Chrome-расширения.

Любой секрет, находящийся в JavaScript клиента, можно извлечь.

Поэтому защита для каждого канала разная.

Web-сервис

Браузер вызывает только Next.js API:

Browser → same-origin Next.js API → internal backend

Conversion Backend вообще не публикуется в интернет.

Защита:

- same-origin cookies;
- CSRF token;
- short-lived request token;
- rate limiting;
- Turnstile/CAPTCHA;
- Origin и Referer checks как дополнительный сигнал;
- internal service authentication;
- private network между Next.js API и worker.

Публичный браузер никогда не знает внутренний API key.

GPTs

GPT Actions поддерживают API-key authentication и OAuth. Для ваших GPTs без пользовательских аккаунтов подходит отдельный API key для каждого GPT.  OpenAI Deve... +1

Page2PDF GPT → key A
Web2PDF GPT → key B
HTML2PDF GPT → key C
Web2PowerPoint GPT → key D

Backend проверяет:

- API key;
- GPT client ID;
- endpoint permissions;
- quota;
- request signature, если поддерживается вашей схемой;
- rate limits;
- формат запроса.

Ключи хранятся в конфигурации GPT Action, а не показываются пользователю.

Но нужно понимать: API key — это контроль доступа, а не абсолютная гарантия, что запрос физически исходит только от интерфейса GPT. При утечке ключ можно использовать отдельно. Поэтому нужны:

- разные ключи;
- ротация;
- квоты;
- аномалия-детекция;
- возможность мгновенного отзыва;
- отдельные endpoint'ы;
- ограничения разрешённых операций.

Chrome-расширение

Без регистрации нельзя надёжно идентифицировать конкретного пользователя.

Кроме того, любой статический секрет в extension bundle извлекается.

Поэтому схема:

```
Extension
  ↓
Request challenge
  ↓
Backend issues short-lived token
  ↓
Extension sends signed job request
```

Но даже такую схему модифицированное расширение можно воспроизвести.

Практичная защита:

- проверка `Origin: chrome-extension://<extension-id>` как дополнительный сигнал;
- allowlist ID опубликованного расширения;
- короткоживущие токены;
- Proof-of-Work или Turnstile для подозрительных запросов;
- строгие rate limits;
- request nonce;
- replay protection;
- подпись body короткоживущим session key;

- Chrome Web Store distribution;
- минимальные разрешения.

`chrome.identity` поддерживает внешние authentication flows, но если регистрации принципиально нет, использовать полноценный пользовательский OAuth только ради API-защиты не следует.

 Chrome for ... +1

Самый безопасный вариант:

- текущая вкладка конвертируется локально;
- серверный backend вызывается расширением только для операций, которые невозможно выполнить локально;
- публичный API защищается от злоупотребления, но не обещается абсолютная криптографическая идентификация расширения.

17. API-слои

Public Edge API

```
|— /api/web/*  
|— /api/extension/*  
└— /api/gpt/*
```

Internal Conversion API

```
|— /internal/jobs  
|— /internal/render  
|— /internal/preview  
└— /internal/files
```

Web API

http

```
POST /api/web/v1/previews  
POST /api/web/v1/conversions  
GET  /api/web/v1/jobs/:id  
GET  /api/web/v1/jobs/:id/preview  
GET  /api/web/v1/jobs/:id/download
```

GPT API

http

```
POST /api/gpt/v1/page-to-pdf  
POST /api/gpt/v1/web-to-pdf
```

```
POST /api/gpt/v1/html-to-pdf
POST /api/gpt/v1/web-to-powerpoint
GET /api/gpt/v1/jobs/:id
GET /api/gpt/v1/jobs/:id/download
```

Extension API

http

```
POST /api/extension/v1/session
POST /api/extension/v1/preview
POST /api/extension/v1/render
GET /api/extension/v1/jobs/:id
```

Все payload'ы валидируются через общие schemas, например Zod.

18. Временные файлы и отсутствие истории

Что не хранится

- аккаунты;
- профили;
- история конвертаций;
- список введённых URL;
- готовые файлы после TTL;
- исходный HTML;
- пользовательские preview после завершения;
- связь между разными сессиями.

Что хранится временно

```
jobId
status
format
mode
createdAt
expiresAt
temporary file locator
technical error code
```

Не сохранять полный URL в Redis, если после начала задания он больше не нужен. Можно держать его только в payload очереди до завершения загрузки страницы.

TTL

Рекомендуемая политика:

Preview model:	30 минут
Preview images:	30 минут
Final PDF/PPTX:	15 минут
Download token:	15 минут
Job metadata:	60 минут
Security rate limit:	24 часа в виде ротационного хеша

После первого скачивания можно удалять файл немедленно, но лучше дать 2–3 повторные попытки в рамках короткого TTL: браузер может оборвать загрузку.

19. Рекомендуемый стек

Web

Next.js App Router
TypeScript
React
next-intl
MDX
Zod
Google Analytics
Google Consent Mode
Cloudflare Turnstile

API и worker

Node.js
Fastify
Playwright
Chromium
BullMQ
Redis
PptxGenJS
pdf-lib или PDFKit
Sharp

Fastify лучше Express для этого backend из-за:

- высокой производительности;
- schema-first validation;
- удобных hooks;
- низкого overhead.

Infrastructure

Vercel или Cloudflare: Next.js frontend
Cloud Run / Fly.io / AWS ECS: API и workers
Redis: managed Redis
Temporary object storage: R2/S3 с lifecycle
Cloudflare WAF/CDN

Для серьёзной нагрузки я бы предпочёл:

Next.js → Vercel
API/Worker → Google Cloud Run
Queue → managed Redis
Temporary files → Cloudflare R2 или S3
WAF → Cloudflare

20. Итоговая схема страниц

```
/[locale]
├─ convert-webpage-to-pdf
├─ convert-webpage-to-powerpoint
├─
├─ chrome-extension
├─ chrome-extension/how-to-use
├─
├─ page2pdf-gpt
├─ web2pdf-gpt
├─ html2pdf-gpt
├─ web2powerpoint-gpt
├─
├─ blog
│   └─ [slug]
├─
├─ updates
├─ changelog
├─
├─ privacy
├─ terms
├─ cookie-policy
└─ security
```

21. Финальное разделение ответственности

Компонент	Ответственность
Next.js	UI, SEO, локализация, блог, инструкции, consent, preview interface
API Gateway	проверка клиента, rate limits, CSRF, API keys, job orchestration
Conversion Worker	Chromium, DOM-анализ, секции, screenshots, PDF, PPTX
Redis	очередь, статусы и временные записи
Temporary Storage	preview и итоговые файлы с TTL
Chrome Extension	текущая вкладка, локальная обработка, связь с backend
GPT Actions	вызов специальных закрытых endpoint'ов
Google Analytics	согласованная аналитика и UTM после consent

Ключевой принцип движка:

Visual mode → максимальная визуальная точность
Editable mode → максимум нативных объектов + raster fallback

А ключевой privacy-принцип:

No accounts
No conversion history
No permanent file storage
Short-lived technical processing only

Такую архитектуру можно начинать как MVP, не меняя фундамент при дальнейшем росте. Самая сложная и важная часть продукта — не сайт и не API, а `page-analyzer + layout engine`, который определяет, какие элементы сохранять нативно, а какие безопасно растривать.

Today 11:16 AM

еще должна быть отдельная страница где описан кейс конвертации чата гпт и отдельная с

конвертацией клада, ну короче для всех
отдельная: гемини, грок
это должны быть отдельные страницы
лендинги где расширение как бы
позиционируется со стороны экспорта чатов в
пдф

Да, это отдельный **SEO-кластер под экспорт AI-чатов**, где основной продукт — именно Chrome-расширение, потому что чаты обычно доступны только внутри авторизованной вкладки пользователя.

Новые посадочные страницы

Добавляем отдельные страницы:

```
/[locale]/export-chatgpt-to-pdf  
/[locale]/export-claude-to-pdf  
/[locale]/export-gemini-to-pdf  
/[locale]/export-grok-to-pdf
```

Позже можно расширить:

```
/[locale]/export-copilot-chat-to-pdf  
/[locale]/export-perplexity-chat-to-pdf  
/[locale]/export-deepseek-chat-to-pdf
```

Для PPTX на первом этапе отдельные страницы, вероятно, не нужны.
Основной понятный пользовательский интент для чатов — именно:

```
Export AI chat to PDF  
Save AI conversation as PDF  
Download ChatGPT conversation  
Print Claude chat without broken formatting
```

Внутри страницы можно упомянуть, что расширение также умеет экспортировать чат в PowerPoint, но основной SEO-интент страницы должен оставаться PDF.

Позиционирование страниц

Это не обычные статьи и не копии общей страницы расширения. Каждая — полноценный продуктовый лендинг под конкретный сценарий.

ChatGPT

Export ChatGPT Conversations to PDF

Save the complete conversation currently open in ChatGPT as a clean PDF.
Preserve messages, code blocks, tables, links and formatting.

Основные преимущества:

- экспорт длинного чата целиком;
- пользовательские и AI-сообщения;
- сохранение code blocks;
- сохранение таблиц;
- кликабельные ссылки;
- разделение на PDF-страницы без случайных разрывов;
- работа с авторизованной вкладкой;
- локальная обработка.

Claude

Export Claude Chats to PDF

Turn a Claude conversation into a clean, readable PDF directly from the open browser

Особенности лендинга:

- длинные ответы Claude;
- артефакты и вложенные блоки;
- код;
- Markdown;
- таблицы;
- ссылки;
- длинные документы в чате;
- корректное деление на страницы.

Gemini

Export Gemini Conversations to PDF

Save your Gemini chat as a structured PDF with readable messages, code blocks and c

Особенности:

- разные типы ответов;
- карточки источников;
- ссылки;
- код;

- таблицы;
- изображения, если они доступны в DOM страницы.

Grok

Export Grok Chats to PDF

Download the conversation currently open in Grok as a clean and shareable PDF.

Особенности:

- ответы и пользовательские сообщения;
- посты и ссылки из X;
- цитируемые источники;
- длинные треды;
- форматирование текста.

Архитектурное отличие от веб-конвертера

AI-чаты нельзя надёжно конвертировать через обычный веб-сервис по URL:

```
chatgpt.com/c/...
claude.ai/chat/...
gemini.google.com/app/...
grok.com/...
```

Причины:

- страницы требуют авторизации;
- URL доступен только владельцу аккаунта;
- контент часто загружается динамически;
- backend не имеет пользовательской сессии;
- отправлять cookies или токены пользователя на сервер нельзя;
- это противоречит privacy-first позиционированию.

Поэтому на таких страницах главным СТА должно быть:

Install the Chrome extension

А не форма ввода URL.

Поток работы

Пользователь открывает ChatGPT / Claude / Gemini / Grok

↓

Открывает расширение

↓

Расширение анализирует текущую вкладку

↓

Определяет платформу и структуру чата

↓

Показывает preview

↓

Пользователь скачивает PDF

Для AI-чатов лучше делать обработку **полностью локально**, без отправки содержимого разговора на backend.

Отдельные адаптеры платформ

В расширении нужен не один универсальный parser, а система адаптеров:

```
packages/  
  chat-export-core/  
    src/  
      adapters/  
        chatgpt.adapter.ts  
        claude.adapter.ts  
        gemini.adapter.ts  
        grok.adapter.ts  
  
      extractors/  
        messages.ts  
        code-blocks.ts  
        tables.ts  
        links.ts  
        images.ts  
  
      renderers/  
        pdf.ts  
        preview.ts
```

Общий интерфейс:

TypeScript

```
interface ChatPlatformAdapter {  
  id: "chatgpt" | "claude" | "gemini" | "grok";  
  
  matches(url: URL, document: Document): boolean;
```

```
detectConversationRoot(): HTMLElement | null;

extractConversation(): Promise<NormalizedConversation>;

loadCompleteConversation(): Promise<void>;
}
```

Нормализованная модель:

TypeScript

```
interface NormalizedConversation {
  title?: string;
  platform: string;
  createdAt?: string;
  messages: ConversationMessage[];
}

interface ConversationMessage {
  role: "user" | "assistant" | "system";
  blocks: ContentBlock[];
}

type ContentBlock =
  | TextBlock
  | CodeBlock
  | TableBlock
  | ImageBlock
  | LinkBlock
  | QuoteBlock;
```

Это позволит не делать четыре независимых экспортера.

Почему адаптеры обязательны

У платформ различаются:

- DOM-структура;
- классы;
- виртуализированные списки;
- lazy-loading старых сообщений;
- структура code blocks;
- таблицы;
- кнопки копирования;
- источники;
- артефакты;
- вложения;

- способы отображения Markdown.

Универсальный алгоритм может использоваться как fallback, но платформенные адаптеры дадут более стабильный результат.

Fallback

```
graph TD
    A[Известная платформа] --> B[Специализированный adapter]
    B --> C[Не удалось извлечь чат]
    C --> D[Generic conversation detector]
    D --> E[Visual export текущей страницы]
```

Два режима для AI-чатов

Для чатов можно сохранить ту же продуктовую модель.

Visual PDF

- чат снимается визуально;
- страницы формируются из скриншотов;
- максимально сохраняется внешний вид платформы;
- ссылки можно наложить как кликабельные области.

Editable PDF

- настоящий текст;
- выделение и копирование;
- кликабельные ссылки;
- настоящие code blocks;
- таблицы;
- отдельные блоки сообщений;
- чистый дизайн Page2File, а не обязательное копирование интерфейса платформы.

Здесь `Editable` лучше позиционировать как:

Clean PDF optimized for reading, searching and copying.

То есть для чатов два режима могут называться:

Original look
Clean document

А внутри сопоставляться с общей системой:

Original look → Visual
Clean document → Editable

Это понятнее обычным пользователям.

Предпросмотр экспорта чата

Предпросмотр строится локально в расширении.

Пользователь может:

- исключить отдельные сообщения;
- убрать системные блоки;
- включить или выключить имя платформы;
- выбрать светлый или оригинальный вид;
- включить заголовок разговора;
- включить дату экспорта;
- выбрать размер страницы;
- увидеть разрывы страниц.

История не сохраняется.

```
DOM чата
  ↓
NormalizedConversation
  ↓
Preview в расширении
  ↓
PDF Blob
  ↓
chrome.downloads.download()
```

Backend для этого сценария вообще не нужен.

Структура каждого лендинга

Все четыре страницы должны использовать общий шаблон, но иметь уникальный контент.

1. Hero

Export ChatGPT Conversations to PDF

Save the complete chat currently open in your browser as a clean, searchable PDF without uploading the conversation to a server.

[[Install Chrome Extension](#)]

2. Демонстрация

- скрин открытого чата;
- окно расширения;
- preview;
- итоговый PDF.

3. Что сохраняется

- сообщения;
- код;
- таблицы;
- ссылки;
- форматирование;
- изображения;
- источники.

Список адаптируется под конкретную платформу.

4. Два режима

Original look

Preserves the visual appearance of the chat.

Clean document

Creates a searchable PDF with selectable text and clickable links.

5. Как пользоваться

1. Открыть разговор.
2. Нажать на расширение.
3. Выбрать PDF.
4. Выбрать режим.
5. Проверить preview.
6. Скачать.

6. Privacy

Your conversation stays in your browser.

The extension reads the currently open tab only after you start an export.
Chat content and the generated PDF are not uploaded to our servers.

Это должно быть правдой технически.

7. Ограничения

Например:

- часть внешних изображений может быть недоступна;
- видео не встраивается;
- закрытые вложения могут экспортироваться только как ссылка;
- платформа может изменить интерфейс;
- очень длинные чаты могут потребовать загрузки предыдущих сообщений.

8. FAQ

Отдельный FAQ для каждой платформы:

Can I export a complete ChatGPT conversation?
Does the exported PDF contain selectable text?
Are code blocks preserved?
Does Page2File upload my conversations?
Can I export a shared ChatGPT link?
Can I export images and tables?

9. Финальный СТА

Export your ChatGPT conversation privately

[Add Page2File to Chrome]

Обновлённая структура сайта

```
/[locale]
├─ convert-webpage-to-pdf
├─ convert-webpage-to-powerpoint
├─
├─ chrome-extension
├─ chrome-extension/how-to-use
├─
├─ export-ai-chat-to-pdf
```

```
|— export-chatgpt-to-pdf
|— export-claude-to-pdf
|— export-gemini-to-pdf
|— export-grok-to-pdf
|
|— page2pdf-gpt
|— web2pdf-gpt
|— html2pdf-gpt
|— web2powerpoint-gpt
|
|— blog
|   └─ [slug]
|
|— updates
|— changelog
|— privacy
|— terms
|— cookie-policy
└─ security
```

Родительская страница

Полезна также общая страница:

```
/[locale]/export-ai-chat-to-pdf
```

Она сравнивает поддерживаемые сервисы:

Platform	Text	Code	Tables	Links	Local export
ChatGPT	Да	Да	Да	Да	Да
Claude	Да	Да	Да	Да	Да
Gemini	Да	Да	Да	Да	Да
Grok	Да	Да	Да	Да	Да

Она связывает дочерние страницы внутренними ссылками.

SEO-структура кластера

```
/export-ai-chat-to-pdf
/      |      \
/      |      \
```

Внутренние ссылки:

- родительская страница ссылается на все платформы;
- каждая платформенная страница ссылается на общую;
- каждая страница ссылается на инструкцию расширения;
- блог может содержать одну сравнительную статью;
- страницы не должны дублировать друг друга слово в слово.

Примеры title:

Export ChatGPT to PDF – Save Complete Conversations
Export Claude Chats to PDF – Private Chrome Extension
Export Gemini Conversations to PDF
Export Grok Chats to PDF

Мультиязычность

Все страницы включаются в общую locale-структуру:

/en/export-chatgpt-to-pdf
/de/export-chatgpt-to-pdf
/fr/export-chatgpt-to-pdf
/es/export-chatgpt-to-pdf
/nl/export-chatgpt-to-pdf

Slug можно оставить английским на всех языках — это упростит маршрутизацию, аналитику и поддержку. Переводятся:

- title;
- description;
- hero;
- преимущества;
- инструкции;
- FAQ;
- metadata;
- structured data.

Важное правило использования брендов

На страницах нужно явно указать:

Page2File is an independent product and is not affiliated with, endorsed by, or sponsored by OpenAI, Anthropic, Google or xAI.

Не использовать оформление так, будто это официальное расширение платформы.

В title и тексте бренды можно использовать для описания совместимости и назначения, но:

- не ставить чужой логотип как основной знак продукта;
- не называть расширение `official ChatGPT PDF Exporter` ;
- не имитировать фирменный интерфейс;
- не использовать формулировки, создающие впечатление партнёрства.

Итоговое дополнение к архитектуре

Появляется ещё один самостоятельный модуль:

Page Conversion Engine

- └─ URL → PDF
- └─ URL → PPTX

Chat Export Engine

- └─ ChatGPT adapter
- └─ Claude adapter
- └─ Gemini adapter
- └─ Grok adapter
- └─ Generic chat fallback

При этом:

- **webpage-конвертация по ссылке** выполняется серверным worker;
- **текущая вкладка и AI-чаты** обрабатываются локально расширением;
- лендинги AI-платформ приводят органический трафик прямо на установку расширения;
- содержимое частных разговоров не отправляется на сервер;
- предпросмотр и итоговый PDF создаются в браузере;
- история экспортов отсутствует.

Это усиливает продукт: Page2File становится не просто конвертером сайтов, а понятным приватным инструментом для сохранения важных AI-разговоров.

ChatGPT can make mistakes. Check important info.